

9 Cross Site Scripting (XSS)

Atak Cross-Site Scripting (XSS) jest jednym z najczęstszych rodzajów ataków w Internecie, który polega na wstrzykiwaniu złośliwego kodu JavaScript do stron internetowych odwiedzanych przez innych użytkowników. Dzięki XSS, atakujący może przejąć kontrolę nad sesjami użytkowników, kraść dane, a także wprowadzać inne formy złośliwego działania.

Reflected XSS

Reflected XSS (zwany także *Non-Persistent XSS*) jest rodzajem ataku, który zachodzi, gdy złośliwy kod JavaScript jest wstrzykiwany w parametrach URL lub w żądaniach HTTP (np. w formularzach). Kod ten zostaje natychmiast przetworzony i odesłany z powrotem do przeglądarki użytkownika w odpowiedzi serwera, a przeglądarka wykonuje go, myśląc, że jest to bezpieczny skrypt. Załóżmy, że mamy aplikację, która przyjmuje dane w parametrze URL, jak np.

`http://example.com/search?q=<script>alert('XSS')</script>`

Jeśli serwer odpowiednio nie sanitizuje danych wejściowych i odsyła je bezpośrednio do przeglądarki, wówczas złośliwy skrypt w parametrze `q` zostanie wykonany w kontekście strony.

Aby zabezpieczyć aplikację przed Reflected XSS, należy:

- **Walidacja i sanitizacja danych wejściowych:** Wszystkie dane pochodzące od użytkownika, w tym parametry URL i dane formularzy, powinny być starannie walidowane i oczyszczane z potencjalnie niebezpiecznych znaków (np. `<`, `>`, `&`, `"`, `'`).
- **Używanie nagłówków Content Security Policy (CSP):** CSP pozwala określić, które źródła mogą ładować skrypty, ograniczając tym samym możliwość wstrzykiwania złośliwego kodu.
- **Escape danych przed wyświetleniem:** Przed wyświetleniem danych na stronie, należy je *escapować* (zamienić specjalne znaki na ich odpowiedniki w HTML).

Stored XSS

Stored XSS (zwany także *Persistent XSS*) zachodzi, gdy złośliwy kod JavaScript jest wstrzykiwany do aplikacji i zapisywany na serwerze, np. w bazie danych, pliku logu lub innym trwałym magazynie danych. Kiedy inny użytkownik odwiedza stronę, która wyświetla te dane, złośliwy skrypt jest automatycznie wykonany przez jego przeglądarkę. Załóżmy, że użytkownik wpisuje w formularzu komentarz, który zawiera złośliwy skrypt:

`<script>alert('XSS')</script>`

Jeśli aplikacja zapisuje ten komentarz w bazie danych bez odpowiedniego filtrowania, a następnie wyświetla go innym użytkownikom, to skrypt zostanie wykonany na komputerze odwiedzającego stronę użytkownika.

Aby zabezpieczyć aplikację przed Stored XSS, należy:

- **Sanitizacja danych przy zapisie:** Wszystkie dane wprowadzane przez użytkowników powinny być sanitizowane przed zapisaniem ich w bazie danych, eliminując wszelkie tagi HTML oraz skrypty JavaScript.
- **Escape danych przy wyświetlaniu:** Zanim dane zostaną wyświetlone w HTML, należy je odpowiednio *escape'ować*, aby zapobiec wykonaniu wstrzykniętych skryptów.
- **Używanie odpowiednich nagłówków HTTP:** Nagłówki takie jak *X-XSS-Protection* mogą pomóc w ochronie przed niektórymi formami XSS, choć nie zastępują one pełnej walidacji i sanitizacji danych.

DOM-based XSS

DOM-based XSS występuje, gdy złośliwy skrypt jest wstrzykiwany do aplikacji przez manipulację Document Object Model (DOM) w przeglądarce, bez konieczności interakcji z serwerem. Złośliwy kod jest uruchamiany w momencie, gdy aplikacja webowa przetwarza dane wejściowe użytkownika bez odpowiedniego oczyszczania, a te dane trafiają do manipulacji DOM.

W przypadku DOM-based XSS, atak może wyglądać tak:

`http://example.com/#q=<script>alert('XSS')</script>`

Aplikacja może następnie wziąć parametr `q` z URL i użyć go do manipulacji DOM, np. wstawiając go jako część treści strony, co skutkuje wykonaniem skryptu w przeglądarce.

Aby zabezpieczyć aplikację przed DOM-based XSS, należy:

- **Walidacja danych wejściowych:** Ważne jest, aby wszystkie dane wejściowe, które mogą być użyte w manipulacji DOM, były odpowiednio walidowane i oczyszczane.

- **Używanie bezpiecznych metod manipulacji DOM:** Zamiast bezpośredniego manipulowania HTML za pomocą takich metod jak `innerHTML`, lepiej używać metod takich jak `textContent` lub `createElement`, które nie interpretują danych jako HTML.
- **CSP i nagłówki bezpieczeństwa:** Używanie polityki Content Security Policy oraz innych nagłówków, takich jak `X-XSS-Protection`, może pomóc w ochronie przed DOM-based XSS.

Ataki XSS stanowią poważne zagrożenie dla bezpieczeństwa aplikacji webowych. Dzielą się one na trzy główne typy: Reflected XSS, Stored XSS oraz DOM-based XSS, z których każdy wymaga innych metod zabezpieczeń. Aby skutecznie chronić aplikacje przed tymi atakami, należy stosować odpowiednią walidację i sanitizację danych wejściowych, korzystać z polityk bezpieczeństwa (takich jak CSP) oraz używać bezpiecznych metod manipulacji DOM. Regularne audyty bezpieczeństwa i testy penetracyjne są również kluczowe w zapewnianiu ochrony przed XSS.

Linki

- <https://sekurak.pl/czym-jest-xss/>
- <https://portswigger.net/burp/documentation/desktop/testing-workflow/input-validation/xss>
- <https://portswigger.net/web-security/cross-site-scripting/cheat-sheet>
- <https://portswigger.net/support/using-burp-to-find-cross-site-scripting-issues>
- <https://medium.com/@kaorrosi/xss-discovery-and-exploitation-with-burpsuite-91d98865c1ee>
- <https://owasp.org/www-community/attacks/xss/>
- <https://book.hacktricks.xyz/pentesting-web/xss-cross-site-scripting>
- https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html
- <https://github.com/payloadbox/xss-payload-list>
- <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/XSS%20Injection>
- <https://github.com/allanlw/svg-cheatsheet>
- <https://book.hacktricks.xyz/pentesting-web/xss-cross-site-scripting/server-side-xss-dynamic-pdf>
- <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- <https://www.geeksforgeeks.org/local-storage-vs-cookies/>
- <https://blog.logrocket.com/localstorage-javascript-complete-guide/>
- <https://codewithpawan.medium.com/enhancing-security-and-efficiency>

Zadania

10.1 Wewnątrz kontenerów działają proste web serwery, które pobierają input od użytkownika i wyświetlają go. Jeden z serwerów posiada zabezpieczenia przed atakiem XSS, drugi nie.

- (a) Korzystając z przygotowanego obrazu Dockerowego `mazurkatarzyna/xss-example-server-1:latest`, spróbuj wykonać atak XSS. Spróbuj wyświetlić zawartość nagłówka `Cookie`. Sprawdź kod źródłowy serwera. W jaki sposób można zmodyfikować kod, aby przeprowadzenie ataku było niemożliwe?

Kontener uruchom za pomocą polecenia: `docker run -it -p 9901:9901 mazurkatarzyna/xss-example-server-1`. Serwer (już wewnątrz kontenera) uruchom za pomocą polecenia `python3 app.py` gdzie `app.py` jest nazwą pliku z serwerem. Po uruchomieniu, serwer będzie działał pod adresem: `http://127.0.0.1:9901`.

- (b) Korzystając z przygotowanego obrazu Dockerowego `mazurkatarzyna/xss-example-server-2:latest`, spróbuj wykonać atak XSS. Spróbuj wyświetlić zawartość nagłówka `Cookie`. Sprawdź kod źródłowy serwera. Czy atak XSS się wykonał? Dlaczego?

Kontener uruchom za pomocą polecenia: `docker run -it -p 9902:9902 mazurkatarzyna/xss-example-server-2`. Serwer (już wewnątrz kontenera) uruchom za pomocą polecenia `python3 app.py` gdzie `app.py` jest nazwą pliku z serwerem. Po uruchomieniu, serwer będzie działał pod adresem: `http://127.0.0.1:9902`.

10.2 Korzystając z przygotowanego obrazu Dockerowego (`mazurkatarzyna/dvwa-v110`), uruchom serwer za pomocą polecenia: `docker run -dp 10102:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 10102, sprawdź w przeglądarce: `http://127.0.0.1:10102`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność XSS. Podpowiedz: w tym zadaniu wykorzystaj Reflected XSS.

Na poziomie `low security` (brak zabezpieczeń):

- Ustaw poziom zabezpieczeń na `Low`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS wyświetl dowolną wiadomość.
- Za pomocą ataku XSS wyświetl ciasteczko użytkownika.

Na poziomie `medium security` (proste zabezpieczenia):

- Ustaw poziom zabezpieczeń na `Medium`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS wyświetl dowolną wiadomość.
- Za pomocą ataku XSS wyświetl ciasteczko użytkownika.

Na poziomie `high security` (lepsze zabezpieczenia):

- Ustaw poziom zabezpieczeń na `High`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS wyświetl dowolną wiadomość.
- Za pomocą ataku XSS wyświetl ciasteczko użytkownika.

10.3 Korzystając z przygotowanego obrazu Dockerowego (`mazurkatarzyna/dvwa-v110`), uruchom serwer za pomocą polecenia: `docker run -dp 10103:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 10103, sprawdź w przeglądarce: `http://127.0.0.1:10103`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność XSS. Podpowiedz: w tym zadaniu wykorzystaj Stored XSS.

Na poziomie `low security` (brak zabezpieczeń):

- Ustaw poziom zabezpieczeń na `Low`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu `stored` wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu `stored` wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu `stored` wyświetl nazwę bieżącej domeny.

Na poziomie `medium security` (proste zabezpieczenia):

- Ustaw poziom zabezpieczeń na `Medium`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu `stored` wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu `stored` wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu `stored` wyświetl nazwę bieżącej domeny.

Na poziomie **high security** (lepsze zabezpieczenia):

- Ustaw poziom zabezpieczeń na **Low**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu stored wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu stored wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu stored wyświetl nazwę bieżącej domeny.

10.4 Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/dvwa-v110), uruchom serwer za pomocą polecenia : `docker run -dp 10103:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 10103, sprawdź w przeglądarce: `http://127.0.0.1:10103`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność XSS. Podpowiedz: w tym zadaniu wykorzystaj DOM XSS.

Na poziomie **low security** (brak zabezpieczeń):

- Ustaw poziom zabezpieczeń na **Low**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu DOM wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu DOM wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu DOM wyświetl nazwę bieżącej domeny.

Na poziomie **medium security** (proste zabezpieczenia):

- Ustaw poziom zabezpieczeń na **Medium**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu DOM wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu DOM wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu DOM wyświetl nazwę bieżącej domeny.